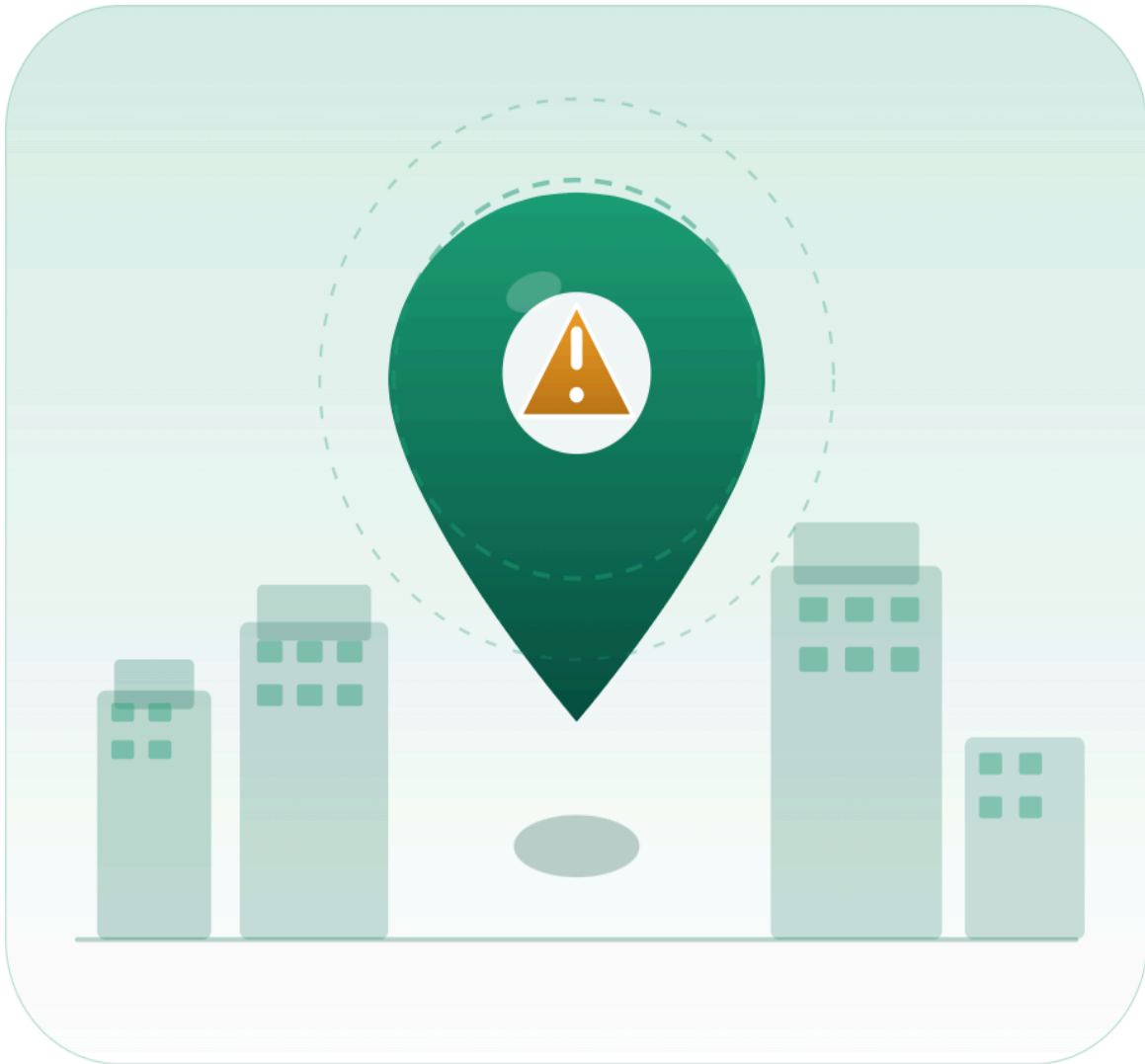


UrbiConnect

Especificación de requisitos y diseño de software

Aplicación de gestión de incidencias ciudadanas en el pueblo de Cantillana



Sergio Algarrada Miranda
21 de Abril de 2026
Versión 1.0 (MVP)

1. DESCRIPCIÓN DEL PROYECTO

- 1.1. Nombre de la aplicación
- 1.2. Objetivo general de la aplicación
- 1.3. Público objetivo
- 1.4. Tipos de incidencias que podrán registrarse

2. ANÁLISIS DE REQUISITOS

- 2.1. Requisitos funcionales
 - 2.1.1. Módulo de autenticación y registro
 - 2.1.2. Módulo del ciudadano
 - 2.1.3. Módulo del responsable municipal
 - 2.1.4. Módulo del administrador
- 2.2. Requisitos no funcionales
 - 2.2.1. Rendimiento
 - 2.2.2. Seguridad
 - 2.2.3. Usabilidad
 - 2.2.4. Disponibilidad y fiabilidad
 - 2.2.5. Mantenibilidad y escalabilidad
 - 2.2.6. Portabilidad y compatibilidad

3. CASOS DE USO

- 3.1. Diagrama de casos de uso
- 3.2. Descripción de al menos tres casos de uso detallados
 - Caso de uso Crear incidencia
 - Caso de uso Cambiar estado de incidencia
 - Caso de uso Iniciar sesión

4. DISEÑO INICIAL DEL SISTEMA

- 4.1. Arquitectura propuesta
- 4.2. Tecnologías elegidas

5. MODELO DE DATOS INICIAL

- 5.1. Diagrama entidad-relación
- 5.2. Descripción de entidades principales

6. BOCETOS DE INTERFAZ (WIREFRAMES)

- Registro/Login
- Envío de incidencias

7. CAMBIOS IMPLEMENTADOS EN LA PRIMERA VERSIÓN

- 7.1. Cambios respecto al diseño inicial
- 7.2. Modelo de datos definitivo
- 7.3. Explicación de la arquitectura implementada
- 7.1. Interfaz de servicios interna

1. DESCRIPCIÓN DEL PROYECTO

1.1. Nombre de la aplicación

El nombre escogido para nuestra aplicación es UrbiConnect. UrbiConnect es una combinación de dos términos modernos que reflejan la esencia de la aplicación:

- *Urbi*: Proviene del latín urbs, urbis, que significa "ciudad" o "entorno urbano".
- *Connect*: Del inglés "conectar", que implica unión, comunicación y colaboración.

UrbiConnect representa una conexión directa y efectiva entre los ciudadanos y la gestión urbana. Es la plataforma que une a la comunidad con las autoridades municipales para reportar y resolver incidencias en la ciudad de forma rápida y transparente.

1.2. Objetivo general de la aplicación

El proyecto tiene como objetivo desarrollar una aplicación multiplataforma (móvil y web) que permita a los ciudadanos comunicar incidencias en la vía pública de manera rápida y sencilla. La aplicación servirá como medio de comunicación entre los vecinos y el ayuntamiento, facilitando la notificación de problemas como farolas rotas, baches o daños en la vía, contenedores llenos o daños en el mobiliario urbano.

Los ciudadanos podrán registrarse, enviar una descripción de la incidencia, adjuntar imágenes y permitir que la app capture su ubicación GPS automáticamente. La información se envía al departamento municipal correspondiente, que podrá gestionar y actualizar su estado (pendiente, en proceso, resuelta). El sistema permite un seguimiento completo, fomentando la transparencia y la participación ciudadana.

1.3. Público objetivo

La aplicación está dirigida principalmente a los ciudadanos de Cantillana y al Ayuntamiento, quienes podrán reportar y gestionar incidencias urbanas de manera eficiente. Se prevé que, en un futuro, la plataforma pueda extenderse a otros municipios, manteniendo la misma funcionalidad y flujo de comunicación.

1.4. Tipos de incidencias que podrán registrarse

En un principio, las distintas categorías en las que se podrán clasificar las incidencias, por tanto las que definen qué tipo de percances pueden reportarse desde la aplicación, son las siguientes:

- **LIMPIEZA**: Esta categoría recoge incidencias relacionadas con la suciedad de las calles, así como problemas con papeleras y contenedores.
- **ALUMBRADO**: Esta categoría abarca percances relacionados con la iluminación urbana, por ejemplo, farolas que no funcionan correctamente.
- **VÍA PÚBLICA**: Esta categoría recoge problemas en el espacio urbano, como agujeros en calles o losas rotas.

- RED DE ALCANTARILLADO: Incidencias como alcantarillas rotas o atascadas, malos olores, etc.
- CONTROL DE PLAGAS: Notificaciones sobre la presencia de animales indeseados. Por ejemplo, ratas o cucarachas.
- JARDINERÍA: Incidencias en parques o jardines, como ramas rotas o árboles caídos.
- TRÁFICO Y SEÑALIZACIÓN: Semáforos que no funcionan, señales de tráfico caídas o borrosas, pasos de peatones dañados...
- MOBILIARIO URBANO: Bancos rotos, marquesinas de bus dañadas, señales informativas deterioradas, farolas ornamentales dañadas.
- OBRAS O INFRAESTRUCTURA EN CONSTRUCCIÓN: Barreras mal señalizadas, zonas peligrosas, calles cortadas sin aviso.

Cómo se ha indicado anteriormente, estas categorías son las que la aplicación albergará en su primera versión, pudiéndose ampliar si así lo precisara el ayuntamiento o el equipo de desarrollo lo considerara oportuno.

2. ANÁLISIS DE REQUISITOS

2.1. Requisitos funcionales

Este apartado describe las funcionalidades que el sistema debe ofrecer a cada uno de los actores que posteriormente definiremos en el diagrama de casos de uso. Los requisitos se identifican con un código único (RF-XX) para facilitar su trazabilidad a lo largo del documento.

LEYENDA DE PRIORIDADES

PRIORIDAD	CRITERIO DE CLASIFICACIÓN
Alta	Imprescindible para el funcionamiento básico en la versión 1. Sin este requisito la app no puede lanzarse.
Media	Importante pero no bloquea el lanzamiento inicial. Se incluirá en próximas versiones.
Baja	Mejora la experiencia de usuario. Puede diferirse a versiones posteriores.

2.1.1. Módulo de autenticación y registro

Gestiona el acceso al sistema para todos los perfiles de usuario. Es la puerta de entrada a la aplicación y condiciona la seguridad de toda la plataforma.

ID	NOMBRE	DESCRIPCIÓN	ACTOR	PRIORIDAD
RF-01	Registro con email	El sistema permite al usuario registrarse introduciendo nombre, apellidos, nombre de usuario, contraseña y correo electrónico.	Ciudadano	Alta
RF-02	Registro con Google	El sistema ofrece la opción de registrarse usando una cuenta de Google como alternativa al registro manual.	Ciudadano	Media
RF-03	Verificación de email	Tras el registro, el sistema envía un correo de verificación. Solo las cuentas verificadas pueden enviar incidencias, reduciendo reportes fraudulentos.	Ciudadano	Alta
RF-04	Inicio de sesión	El usuario puede autenticarse con su nombre de usuario y contraseña.	Todos	Alta
RF-05	Inicio de sesión con Google	El usuario puede autenticarse opcionalmente usando su cuenta de Google.	Ciudadano	Media
RF-06	Cierre de sesión	El usuario puede cerrar su sesión activa en cualquier momento desde la aplicación.	Todos	Alta

2.1.2. Módulo del ciudadano

Funcionalidades disponibles para los ciudadanos registrados y verificados. Constituyen el núcleo de participación ciudadana de la aplicación.

ID	NOMBRE	DESCRIPCIÓN	ACTOR	PRIORIDAD
RF-07	Crear incidencia	El ciudadano puede reportar una nueva incidencia incluyendo: descripción, fotografía, geolocalización GPS automática, fecha y tipo de incidencia seleccionado de una lista de categorías.	Ciudadano	Alta
RF-08	Ver incidencias propias	El ciudadano puede consultar el listado de incidencias que ha	Ciudadano	Alta

		reportado, junto con su estado actualizado.		
RF-09	Notificaciones push	El ciudadano recibe notificaciones push en el dispositivo móvil cuando cambia el estado de alguna de sus incidencias.	Ciudadano	Alta
RF-10	Notificaciones en app	El ciudadano recibe notificaciones dentro de la aplicación (buzón interno) sobre actualizaciones de sus incidencias.	Ciudadano	Alta
RF-11	Chat con el responsable municipal	El ciudadano puede enviar mensajes al responsable municipal asignado a su incidencia para solicitar información o aclaraciones.	Ciudadano	Media
RF-12	Enviar recordatorio	El ciudadano puede enviar un recordatorio al ayuntamiento sobre una incidencia pendiente de resolución desde hace tiempo.	Ciudadano	Baja

2.1.3. Módulo del responsable municipal

Funcionalidades del personal municipal encargado de gestionar y resolver las incidencias asignadas por el administrador.

ID	NOMBRE	DESCRIPCIÓN	ACTOR	PRIORIDAD
RF-13	Ver incidencias asignadas	El responsable municipal puede consultar el listado completo de incidencias que le han sido asignadas, con todos sus detalles.	Responsable municipal	Alta
RF-14	Cambiar estado de incidencias	El responsable puede actualizar el estado de una incidencia entre los valores: <i>pendiente</i> , <i>en proceso</i> y <i>resuelta</i> .	Responsable municipal	Alta
RF-15	Chat con el ciudadano	El responsable puede comunicarse con el ciudadano que reportó la incidencia para recabar información adicional o informar de la evolución.	Responsable municipal	Media

2.1.4. Módulo del administrador

Funcionalidades de configuración y supervisión global del sistema, reservadas al perfil con máximos privilegios.

ID	NOMBRE	DESCRIPCIÓN	ACTOR	PRIORIDAD
RF-16	Ver usuarios	El administrador puede consultar el listado completo de usuarios registrados en el sistema.	Administrador	Alta
RF-17	Eliminar usuarios	El administrador puede eliminar cuentas de usuarios del sistema cuando sea necesario.	Administrador	Alta
RF-18	Crear responsable municipal	El administrador puede crear cuentas con rol de responsable municipal, asignándoles los tipos de incidencias de los que serán responsables.	Administrador	Alta
RF-19	Gestionar categorías	El administrador puede crear, editar y eliminar las categorías de incidencias disponibles en la aplicación	Administrador	Alta

2.2. Requisitos no funcionales

Los requisitos no funcionales (RNF) definen las cualidades y restricciones que debe cumplir el sistema más allá de sus funcionalidades. Determinan cómo se comporta la aplicación en términos de rendimiento, seguridad, usabilidad, disponibilidad y mantenibilidad.

2.2.1. Rendimiento

El sistema debe responder con agilidad para garantizar una experiencia fluida tanto en la app móvil como en el panel web, incluso bajo carga concurrente de usuarios.

ID	NOMBRE	DESCRIPCIÓN	CATEGORÍA	PRIORIDAD
RNF-01	Tiempos de respuesta	Las peticiones a la base de datos y la carga de pantallas deben resolverse en un tiempo máximo de 3 segundos bajo una conexión estándar.	Rendimiento	Alta
RNF-02	Optimización de imágenes	Las fotografías se comprimirán automáticamente en el dispositivo antes de subirse a la nube, limitando su tamaño a un máximo de 1 MB por imagen.	Rendimiento	Media

2.2.2. Seguridad

Dado que la aplicación gestiona datos personales de ciudadanos, la seguridad es un requisito crítico que debe garantizarse en todas las capas del sistema.

ID	NOMBRE	DESCRIPCIÓN	CATEGORÍA	PRIORIDAD
RNF-03	Seguridad y cifrado	Las contraseñas no se almacenarán en texto plano. Se utilizará Firebase Auth y conexiones seguras (HTTPS) para proteger los datos en tránsito.	Seguridad	Alta

RNF-04	Control de acceso (reglas)	El sistema implementará reglas estrictas en Firestore para garantizar que los ciudadanos sólo interactúen con sus propios datos, respetando roles.	Seguridad	Alta
--------	----------------------------	--	-----------	------

2.2.3. Usabilidad

La aplicación está destinada a un público amplio y heterogéneo. La interfaz debe ser intuitiva y accesible para cualquier ciudadano, independientemente de su nivel tecnológico.

ID	NOMBRE	DESCRIPCIÓN	CATEGORÍA	PRIORIDAD
RNF-05	Diseño responsive	La interfaz se adapta fluidamente a diferentes resoluciones: móviles, tabletas y navegadores de escritorio, siguiendo Material Design.	Usabilidad	Alta
RNF-06	Experiencia intuitiva	El proceso para reportar una nueva incidencia no debe superar los 5 pasos desde el inicio, garantizando un uso sin necesidad de conocimientos previos.	Usabilidad	Alta
RNF-07	Accesibilidad básica	La aplicación asegurará un contraste de colores suficiente, tamaños de fuente legibles y compatibilidad básica con lectores de pantalla.	Usabilidad	Baja

2.2.4. Disponibilidad y fiabilidad

Al tratarse de un servicio público de comunicación con el Ayuntamiento, la disponibilidad continua de la plataforma es fundamental para mantener la confianza ciudadana.

ID	NOMBRE	DESCRIPCIÓN	CATEGORÍA	PRIORIDAD
RNF-08	Tolerancia a fallos de red	Ante pérdida de conexión, la app informa al usuario con un mensaje claro y permite reintentar la operación sin pérdida de los datos introducidos.	Disponibilidad	Media

2.2.5. Mantenibilidad y escalabilidad

El sistema debe estar diseñado para facilitar su evolución futura, permitiendo añadir nuevas funcionalidades o adaptarlo a otros municipios con el mínimo esfuerzo.

ID	NOMBRE	DESCRIPCIÓN	CATEGORÍA	PRIORIDAD
RNF-09	Mantenibilidad (Gitflow)	El código fuente estará estructurado de forma modular y se gestionará mediante un repositorio Git utilizando la metodología Gitflow.	Mantenibilidad	Media

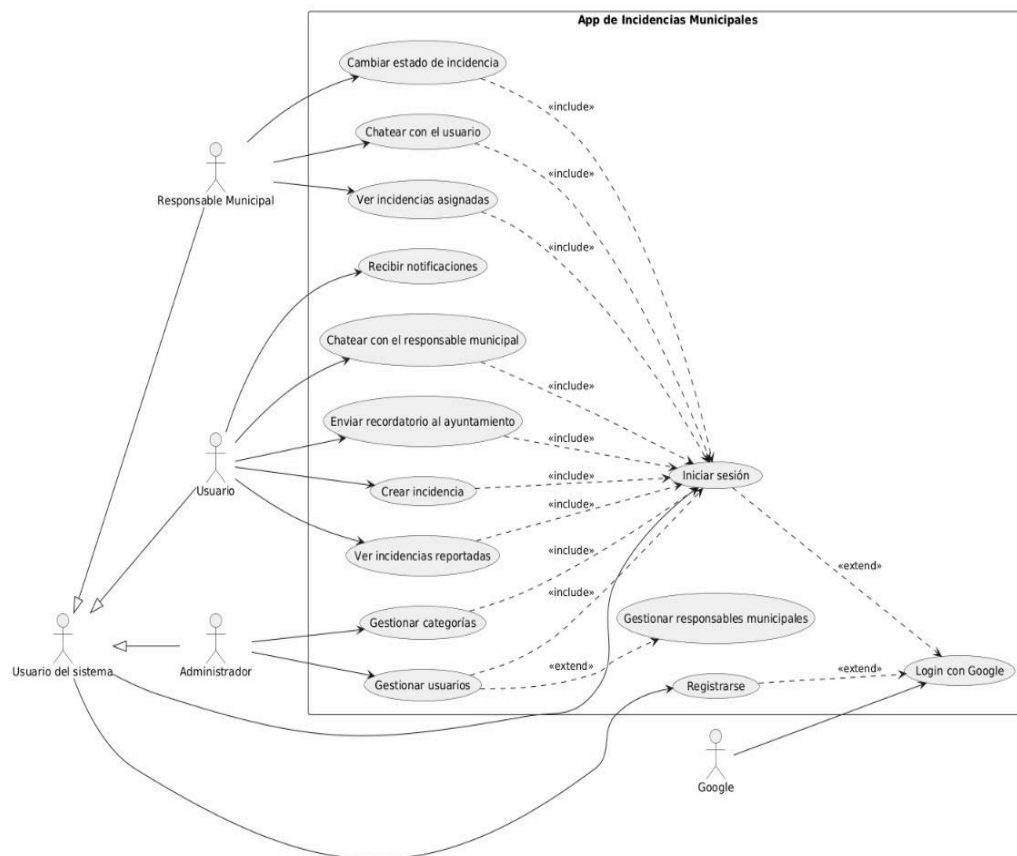
2.2.6. Portabilidad y compatibilidad

La elección de Flutter como tecnología principal garantiza de forma intrínseca la portabilidad de la aplicación entre plataformas con un único código base.

ID	NOMBRE	DESCRIPCIÓN	CATEGORÍA	PRIORIDAD
RNF-10	Compatibilidad multiplataforma	El cliente móvil será compatible con Android 8.0+ e iOS 14+, mientras que el panel web funcionará en versiones recientes de navegadores.	Portabilidad	Alta

3. CASOS DE USO

3.1. Diagrama de casos de uso



3.2. Descripción de al menos tres casos de uso detallados

Caso de uso Crear incidencia

- Actores involucrados: Usuario (Ciudadano).
- Precondición: El usuario debe haber iniciado sesión en la aplicación y tener su cuenta verificada.
- Flujo básico:
 - El usuario solicita reportar una nueva incidencia seleccionando la opción correspondiente en la interfaz.
 - Se pide al usuario la introducción de una breve descripción, la selección de un tipo de incidencia de una lista, adjuntar una fotografía y permitir la captura de su geolocalización GPS.
 - Tras la introducción de los datos, el sistema guarda la incidencia, la envía al departamento municipal correspondiente y muestra un mensaje de confirmación en pantalla.
 - Se vuelve a solicitar al usuario la selección de una operación (volviendo a la pantalla principal o al buzón).
- Caminos alternativos:
 - Si en el paso 3 se detecta una pérdida de conexión a internet, el sistema informa al usuario con un mensaje claro, permite reintentar la operación sin pérdida de los datos introducidos y se pasa al paso 3.
- Poscondición: El sistema dispone de una nueva incidencia registrada con estado "pendiente" que aparece en el listado del ciudadano.

Caso de uso Cambiar estado de incidencia

- Actores involucrados: Responsable municipal.
- Precondición: El responsable debe haber iniciado sesión y tener incidencias asignadas a su departamento.
- Flujo básico:
 - El responsable municipal solicita consultar los datos de una incidencia de su lista de incidencias asignadas.
 - Se muestran en pantalla los detalles de la incidencia y se pide al responsable la selección del nuevo estado (pendiente, en proceso o resuelta).
 - El sistema actualiza el estado en la base de datos y envía una notificación (push o dentro de la app) al ciudadano que la reportó.
 - Se vuelve a solicitar al responsable la selección de una operación en su listado.
- Caminos alternativos:
 - Si en el paso 2 se selecciona el mismo estado en el que ya se encontraba la incidencia, el sistema no realiza ninguna modificación en la base de datos y se pasa al paso 4.
- Poscondición: La incidencia actualiza su estado en el sistema y el ciudadano queda informado del cambio.

Caso de uso Iniciar sesión

- Actores involucrados: Usuario del sistema (Ciudadano, Responsable municipal o Administrador).
- Precondición: El usuario tiene que estar previamente registrado en el sistema.
- Flujo básico:
 - El usuario solicita iniciar sesión en la aplicación.
 - Se pide al usuario la introducción de su nombre de usuario (o correo electrónico) y su contraseña.
 - El sistema comprueba que las credenciales son correctas y permite el acceso al panel correspondiente según el rol del usuario.
 - Se solicita al usuario la selección de una operación.
- Caminos alternativos:
 - Si en el paso 2 el usuario selecciona la opción "Inicio de sesión con Google", el sistema solicita la autenticación a través de la cuenta de Google del usuario y se pasa al paso 3.
 - Si en el paso 3 el sistema detecta que el correo y la contraseña no coinciden, se muestra un mensaje de error advirtiendo que las credenciales son incorrectas y se pasa al paso 2 anterior.
- Poscondición: El usuario tiene una sesión activa en la aplicación y dispone de acceso a las funcionalidades de su rol.

4. DISEÑO INICIAL DEL SISTEMA

4.1. Arquitectura propuesta

El sistema utilizará una arquitectura basada en la nube de tipo Serverless (sin servidor propio) orientada a servicios. En este modelo, las aplicaciones cliente (la aplicación móvil para ciudadanos y el panel web administrativo) se comunican directamente con una plataforma de servicios gestionados en la nube (Backend as a Service) a través de sus SDKs y peticiones seguras HTTPS. Esto permite delegar la infraestructura, el almacenamiento y la escalabilidad de la base de datos al proveedor en la nube, eliminando la necesidad de mantener servidores locales.

4.2. Tecnologías elegidas

Frontend (Aplicación Móvil y Panel Web): Flutter (Dart)

Justificación: La elección de Flutter como tecnología principal garantiza la portabilidad de la aplicación, permitiendo desarrollar el cliente móvil (compatible con Android 8.0+ e iOS 14+) y el panel web utilizando un único código base. Esto reduce drásticamente los tiempos de desarrollo y facilita la mantenibilidad. Además, se empleará Material Design para asegurar un diseño responsive, intuitivo y accesible.

Backend, Base de Datos y Autenticación: Firebase

Justificación: Firebase es la mejor opción para este proyecto porque ofrece un ecosistema unificado de herramientas listas para integrarse con Flutter. Al ser una solución *serverless*, el mantenimiento de la infraestructura es mínimo. Además, su plan gratuito cubre de sobra el volumen estimado inicial (aproximadamente 4000 usuarios y 15 incidencias al día), lo que supone un coste muy bajo o nulo. Se utilizarán los siguientes servicios específicos:

- Firestore: Base de datos NoSQL en tiempo real para almacenar la información de los usuarios y las incidencias.
- Firebase Authentication: Para gestionar el registro y el inicio de sesión seguro (mediante email/contraseña y Google).
- Firebase Storage: Almacenamiento en la nube optimizado para guardar las fotografías de las incidencias.
- Firebase Cloud Messaging: Para la gestión y envío de notificaciones push a los dispositivos de los ciudadanos.

Geolocalización y Mapas: Google Maps API + Geolocator

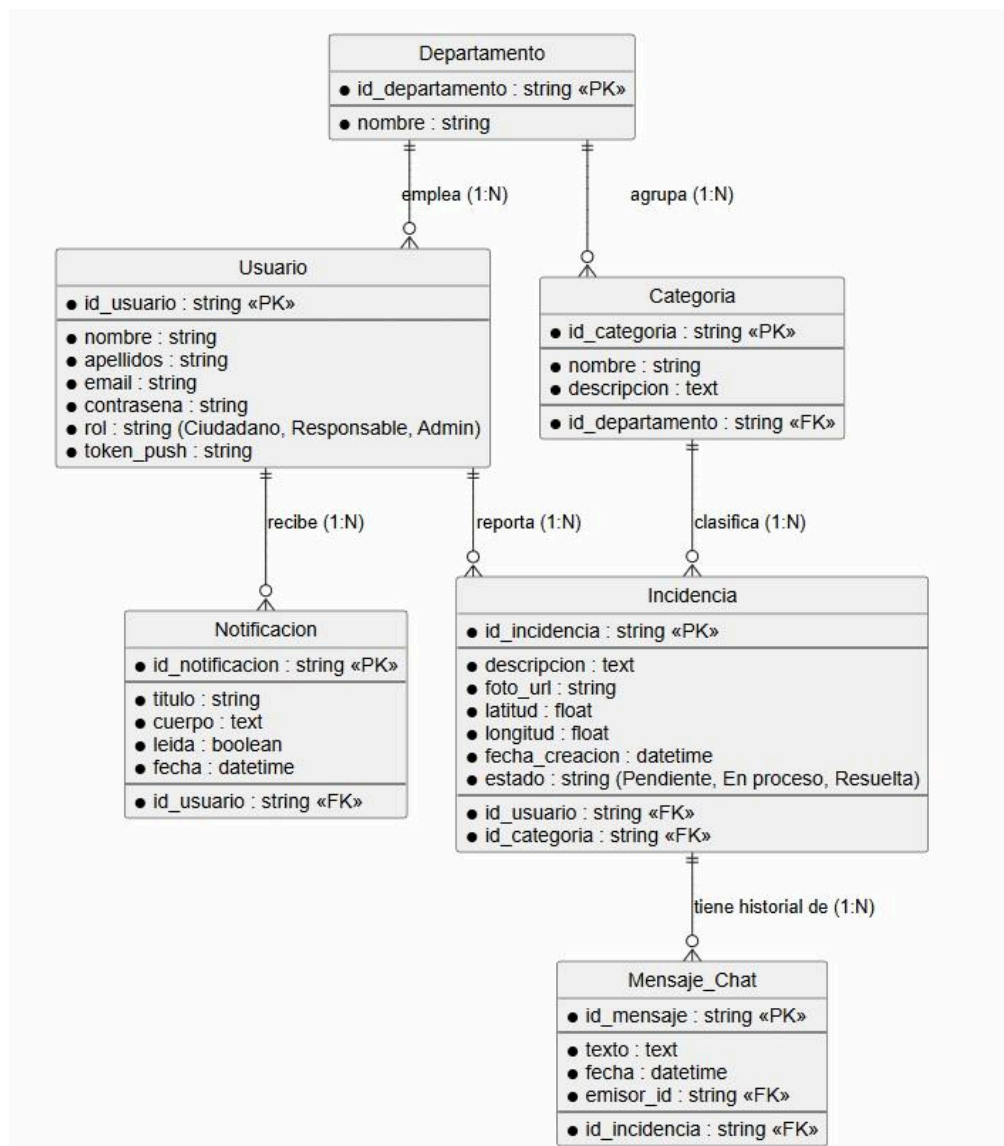
Justificación: Son herramientas fiables y estandarizadas que permiten tanto la captura precisa de las coordenadas GPS en el dispositivo del ciudadano como su posterior visualización en un mapa.

Control de versiones: Git y GitHub

Justificación: El código fuente se gestionará de forma segura y estructurada mediante repositorios en la nube, aplicando la metodología Gitflow para asegurar un desarrollo ordenado y escalable.

5. MODELO DE DATOS INICIAL

5.1. Diagrama entidad-relación



5.2. Descripción de entidades principales

- **Usuario:** Es la entidad que centraliza a todas las personas que acceden al sistema. Para evitar duplicidad de tablas, utiliza un atributo "rol" que diferencia los privilegios de acceso entre Ciudadano, Responsable Municipal y Administrador. Almacena datos básicos como nombre, email y contraseña (o vinculación con Google), así como el *token push* necesario para enviar notificaciones al dispositivo del usuario.

- **Incidencia:** Es la entidad central y el núcleo de la aplicación. Registra toda la información del problema reportado en la vía pública: una descripción textual, la URL de la fotografía (almacenada en Firebase Storage), las coordenadas geográficas exactas (latitud y longitud), la fecha

de creación y su estado actual (Pendiente, En proceso, Resuelta). Se relaciona mediante claves foráneas con el ciudadano que la reporta y con la categoría a la que pertenece.

- **Categoría:** Define los distintos tipos de percances que se pueden reportar en el municipio (por ejemplo, Limpieza, Alumbrado, Vía Pública, etc.). Permite clasificar las incidencias de forma organizada y se vincula a un departamento municipal específico encargado de su gestión.

- **Mensaje_Chat:** Sustenta el requisito de comunicación directa entre el ciudadano y el responsable municipal. Almacena el contenido del mensaje, la fecha y el emisor. Se vincula obligatoriamente a una Incidencia específica para mantener el contexto de la conversación.

- **Notificación:** Da soporte al requisito de avisos y al buzón interno de la aplicación. Registra el título, el cuerpo del mensaje, la fecha y un indicador booleano de lectura, asegurando que el ciudadano pueda consultar el historial de actualizaciones sobre sus incidencias reportadas.

(SE HA ELIMINADO LA COLECCIÓN DEPARTAMENTO DURANTE EL DESARROLLO DE LA APLICACIÓN)

6. BOCETOS DE INTERFAZ (WIREFRAMES)

Registro/Login

+-----+	
UrbiConnect	
(Logo de la app)	
[Correo electrónico	]
[Contraseña	]
(INICIAR SESIÓN	)
¿No tienes cuenta?	
(REGISTRARSE	)
----- O entra con -----	
([G] Continuar con Google	)
+-----+	

Envío de incidencias

+-----+	
[<-]	Nueva Incidencia
+-----+	
1. Categoría del problema:	
[Seleccionar (Limpieza, Vía...) v]	
2. Descripción:	
[Escribe aquí los detalles...]	
[]	
3. Fotografía del desperfecto:	
[[O] Tomar foto o subir imagen]	
4. Ubicación:	
(•) Capturar mi ubicación GPS actual	
(ENVIAR INCIDENCIA)	
+-----+	

Listado de incidencias

+-----+		
Mis Incidencias	[+]	
+-----+		
[Foto]	Farola rota	
	Calle Mayor, 12	
	Estado: EN PROCESO	[>]

[Foto]	Baches en la vía	
	Av. Constitución, 45	
	Estado: PENDIENTE	[>]

[Foto]	Contenedor lleno	
	Plaza de España	
	Estado: RESUELTA	[>]
=====		
[Inicio]	[Buzón]	[Perfil]
+-----+		

7. CAMBIOS IMPLEMENTADOS EN LA PRIMERA VERSIÓN

7.1. Cambios respecto al diseño inicial

Durante la fase de codificación, se realizaron ajustes sobre el diseño original (v0.1) para mejorar la eficiencia del sistema y la experiencia de usuario:

- **Sincronización en tiempo real:** Se ha implementado un flujo de datos dinámico mediante los listeners de Firebase. Esto permite que el estado de las incidencias y los mensajes del chat se actualicen instantáneamente en la interfaz sin necesidad de recargar la aplicación.
- **Gestión de notificaciones y mantenimiento:** Se añadió una lógica de limpieza automática que elimina los registros obsoletos en la colección de notificaciones. Este cambio optimiza el rendimiento de la base de datos y mejora la usabilidad del buzón del ciudadano.
- **Seguridad Basada en Roles (RBAC):** La seguridad se ha reforzado mediante las Reglas de Seguridad de Firestore, garantizando que cada usuario (Ciudadano, Responsable o Admin) solo pueda leer o escribir los datos que le corresponden según su rol definido en el sistema.

7.2. Modelo de datos definitivo

El modelo de datos final se ha implementado en Cloud Firestore siguiendo una estructura documental, organizada en las siguientes colecciones principales:

- **users:** Almacena la información de los usuarios registrados, incluyendo su nombre, apellidos, correo, la fecha de registro, el rol asignado (fundamental para los permisos) y los tokens necesarios para la recepción de mensajes.
- **Incidencia:** Es el núcleo de la aplicación. Cada documento contiene la descripción del problema, las coordenadas geográficas, la fecha de creación, el estado (Pendiente, En proceso, Resuelta) y los enlaces a las imágenes almacenadas en Storage.
- **Categoría:** Colección que define los tipos de incidencias disponibles (Limpieza, Alumbrado, Vía Pública, etc.), permitiendo que el sistema sea escalable si se desea añadir nuevos tipos en el futuro.
- **Mensajes_Chat:** Gestiona la comunicación directa. Cada mensaje queda registrado con su emisor, fecha y contenido, vinculándose a una incidencia específica para mantener el hilo de la conversación.
- **Notificaciones:** Almacena los avisos generados por el sistema (cambios de estado, mensajes nuevos) para que el ciudadano pueda consultarlos en su historial personal.

7.3. Explicación de la arquitectura implementada

La aplicación sigue una arquitectura de tipo BaaS (Backend as a Service), delegando la gestión del servidor y la base de datos a la infraestructura de Firebase:

- **Entorno Multiplataforma (Flutter):** Se utiliza una única base de código para generar tanto la versión Web (PWA) desplegada en el servidor como la versión nativa para Android (APK).
- **Infraestructura en la nube (Firebase):** Gestiona la autenticación segura, la persistencia de datos y el almacenamiento de archivos multimedia.
- **Alojamiento y Despliegue:** La versión web está alojada en un servidor con Plesk Obsidian. Se ha configurado el servidor Apache con certificados SSL para garantizar que la aplicación funcione siempre bajo el protocolo HTTPS, lo cual es obligatorio para el correcto funcionamiento del GPS y la cámara en la web.

7.1. Interfaz de servicios interna

En lugar de una API REST externa, la comunicación se gestiona a través de una capa de servicios en Dart que abstrae la complejidad de Firebase:

- **Servicio de Autenticación (AuthService):** Controla el inicio de sesión con Google y el registro de perfiles.
- **Servicio de Datos (IncidentService):** Centraliza las operaciones de lectura y escritura sobre las colecciones Incidencia y Categoría, aplicando filtros por usuario o estado.
- **Servicio de Mensajería y Chat:** Gestiona la escucha activa de la colección Mensajes_Chat para permitir conversaciones en vivo.⁷
- **Puntos de acceso móvil y web:** El sistema detecta automáticamente la plataforma del usuario para adaptar funciones como el selector de fotos o la precisión del GPS.